# Day 6 - Notes

# Platform Events, Event-Driven Architecture, and Command-Line Interface

## 1. Event-Driven Software Architecture

### What is Event-Driven Architecture?

Event-driven architecture is a software design pattern where systems communicate through events. Instead of directly requesting information from another system, one system publishes an event and other systems subscribe to receive it.

This model follows the Publisher-Subscriber (Pub/Sub) pattern.

- Publisher → Sends event messages
- Subscriber → Receives event messages
- Event Bus → Stores and delivers events

Example:
When an order is shipped, the order system publishes an event. Other systems such as inventory, notifications, or customer support receive the event and react automatically.

---

# Components of Event-Driven Systems

## Event

An important change in a business process.
Example:

- Order placed
- Order shipped
- Payment completed

## Event Message

A message containing information about the event.
Example:

An order shipment event may contain:

- Order number
- Shipment status
- Customer details

## Event Producer

The system or application that publishes the event.
Example:

- Order Management App
- Shipping System

## Event Consumer

The system that subscribes to and receives event messages.
Example:

- Customer notification app
- Salesforce app

## Event Channel

A stream through which events are published and consumed.

## Event Bus

A service that stores and delivers event messages.
Salesforce Event Bus:

- Stores events temporarily
- Delivers events in order
- Supports replay functionality

---

# Benefits of Event-Driven Architecture

## Decoupling

Event producers and consumers are independent.

## Real-Time Communication

Systems react quickly when events occur.

## Scalability

Multiple subscribers can receive the same event.

## Simplified Communication

No need for constant polling or direct integration.

## Flexibility

Supports internal and external integrations.

---

# Real-World Use Cases of Platform Events

## Platform to External App

### Order Fulfillment

When an opportunity becomes Closed Won:

- Salesforce publishes an event
- Vendor shipping app receives the event
- Shipment is processed automatically

## External App to Salesforce

### Merchandise Return

External return system publishes an event.
Salesforce receives the event and:

- Creates follow-up records
- Sends emails
- Updates customer service

## Platform to Platform

### Lead Reassignment

A trigger publishes an event.
Another Salesforce process receives the event and reassigns the lead.

---

# Platform Events

## Definition

Platform Events are special Salesforce objects used for event-driven communication.
They allow apps to:

- Publish events
- Subscribe to events
- Communicate asynchronously

---

# Characteristics of Platform Events

## Similar to Salesforce Objects

Platform events have:

- Fields
- API names
- Permissions

## Differences from Normal Objects

Event messages:

- Cannot be edited
- Cannot be deleted
- Cannot be viewed in UI
- Cannot be queried with SOQL/SOSL

## Supported Field Types

- Checkbox

- Date
- Date/Time
- Number
- Text
- Long Text Area

---

# ReplayId

Each platform event contains a ReplayId field.
Purpose:

- Identifies the position of the event in the stream
- Helps subscribers retrieve missed events

Important:
Replay IDs are not always sequential.

---

# Event Retention

## High-Volume Events

Stored for 72 hours.

## Standard-Volume Events

Stored for 24 hours.
Note:
New platform events are high-volume by default.

---

# API Name Suffix

Platform Events use the suffix:

__e

Example:
Cloud_News**e**
**Order_Event**e

---

# Publish Behavior

## Publish Immediately

Default behavior.

- Event publishes immediately
- Event publishes even if transaction fails

Use Case:
Logging or independent systems.

## Publish After Commit

Event publishes only if transaction succeeds.

Use Case:
When subscribers depend on committed data.

---

# Creating a Platform Event

## Steps

1. Setup
2. Search Platform Events
3. Click New Platform Event
4. Enter details
5. Save
6. Add custom fields

---

# Example: Cloud News Platform Event

## Event Details

Label: Cloud News
Object Name: Cloud_News
Description: Cloud news events deliver news at your fingertips
Publish Behavior: Publish Immediately

## Fields

### Location

Type: Text
Length: 100
API Name: Location__c

### Urgent

Type: Checkbox
API Name: Urgent__c

### News Content

Type: Long Text Area
API Name: News_Content__c

---

# Publishing Platform Events Using Apex

## Basic Syntax

```
Cloud_News__e newsEvent = new Cloud_News__e(
    Location__c='Mountain City',
    Urgent__c=true,
    News_Content__c='Lake Road is closed due to mudslides.'
);

Database.SaveResult sr = EventBus.publish(newsEvent);
```

---

# Checking Publish Result

```
if (sr.isSuccess()) {
    System.debug('Successfully published event.');
} else {
    for(Database.Error err : sr.getErrors()) {
        System.debug(err.getMessage());
    }
}
```

---

## Publishing Multiple Events

```
List<Cloud_News__e> newsEventList = new List<Cloud_News__e>();

Cloud_News__e newsEvent1 = new Cloud_News__e(
    Location__c='Mountain City',
    Urgent__c=true,
    News_Content__c='Mudslides.'
);

Cloud_News__e newsEvent2 = new Cloud_News__e(
    Location__c='Mountain City',
    Urgent__c=false,
    News_Content__c='Traffic issue.'
);

newsEventList.add(newsEvent1);
newsEventList.add(newsEvent2);

List<Database.SaveResult> results = EventBus.publish(newsEventList);
```

---

## EventBus.publish()

### Important Points

- Publishes asynchronously
- Returns Database.SaveResult
- Does not throw exceptions for publish failures

- Supports partial success

---

# Publishing Events Using Flow Builder

Use Create Records element.

Steps:

1. Create Flow
2. Add Create Records
3. Select Platform Event object
4. Set field values
5. Save and activate

Boolean values use:

```
{!$GlobalConstant.True}
```

---

# Publishing Events Using REST API

## REST Endpoint

```
/services/data/vXX.X/sobjects/Cloud_News__e/
```

## Request Body

```
{
  "Location__c": "Mountain City",
  "Urgent__c": true,
  "News_Content__c": "Lake Road is closed due to mudslides."
}
```

## Response

```
{
  "id": "e00xx0000000001AAA",
  "success": true,
  "errors": []
}
```

---

## Subscribing to Platform Events

Platform events can be subscribed using:

- Apex Triggers
- Flows
- Processes
- Lightning Components
- Pub/Sub API

---

## Platform Event Apex Triggers

Platform event triggers support only:

```
after insert
```

---

## CloudNewsTrigger Example

```
trigger CloudNewsTrigger on Cloud_News__e (after insert) {

    List<Case> cases = new List<Case>();

    Group queue = [SELECT Id FROM Group
                   WHERE Name='Regional Dispatch'
                   AND Type='Queue'];

    for (Cloud_News__e event : Trigger.New) {
```

```
        if (event.Urgent__c == true) {

            Case cs = new Case();
            cs.Priority = 'High';
            cs.Subject = 'News team dispatch to ' +
                         event.Location__c;
            cs.OwnerId = queue.Id;

            cases.add(cs);
        }
    }

    insert cases;
}
```

---

# Important Trigger Concepts

## Asynchronous Execution

Triggers run separately from the publishing transaction.

## Automated Process User

Triggers run as:

```
Automated Process
```

Therefore:

- CreatedById becomes Automated Process
- Explicit OwnerId assignment is recommended

## Trigger Batch Size

Default maximum:

```
2000 events
```

---

# Debug Logs for Platform Event Triggers

Steps:

1. Setup
2. Debug Logs
3. New
4. Traced Entity Type = Automated Process
5. Select Debug Level
6. Save

---

# Managing Trigger Subscriptions

From Platform Event detail page:
Subscriptions related list allows:

- Suspend
- Resume
- Resume from Tip

---

# Testing Platform Event Triggers

## Important

Publishing must occur inside:

```
Test.startTest();
Test.stopTest();
```

---

# Test Class Example

```
@isTest
public class PlatformEventTest {
```

```
    @isTest
    static void test1() {

        Cloud_News__e newsEvent = new Cloud_News__e(
            Location__c='Mountain City',
            Urgent__c=true,
            News_Content__c='Test message.'
        );

        Test.startTest();

        Database.SaveResult sr = EventBus.publish(newsEvent);

        Test.stopTest();

        System.assertEquals(true, sr.isSuccess());

        List<Case> cases = [SELECT Id FROM Case];

        System.assertEquals(1, cases.size());
    }
}
```

---

# Lightning Web Component Subscription

## Import empApi

```
import {
  subscribe,
  unsubscribe,
  onError,
  setDebugFlag,
  isEmpEnabled
} from 'lightning/empApi';
```

---

# Aura Component Subscription

```
<lightning:empApi aura:id="empApi"/>
```

---

# Flow Subscription to Platform Events

Use:

```
Platform Event Triggered Flow
```

Use:

```
$Record
```

to access event fields.

---

# Pub/Sub API

## Features

- Publish and subscribe in one API
- High performance
- Supports replay
- Supports multiple languages
- Uses gRPC and HTTP/2

## Supported Languages

- Java
- Python
- Node.js
- C++
- Others

---

# Command-Line Interface (CLI)

## Definition

CLI is a text-based interface used to interact with software and operating systems.
Instead of buttons and menus, users type commands.

---

# GUI vs CLI

## GUI

Graphical User Interface
Examples:

- Buttons
- Menus
- Screens

## CLI

Command-Line Interface
Uses typed commands.

---

# Advantages of CLI

- Faster execution
- Automation support
- Script execution
- Better for developers
- Supports integrations
- Package management

---

# Common Command-Line Tools

## macOS/Linux

Terminal

## Windows

Command Prompt
or
PowerShell

## Code Editor Terminal

VS Code terminal is recommended.

---

# Shell

A shell interprets and executes commands.

---

# Common Shells

## PowerShell

Common in Windows.

## ZSH (Z Shell)

Common in macOS/Linux.

## BASH

Bourne Again Shell.

---

# BASH

## Meaning

Bourne Again Shell

## Features

- Command execution
- Script support
- Widely used in development

---

# Command Structure

General format:

```
command -flag arguments
```

---

# Parts of a Command

## Command

Specifies the action.

Example:

```
sf project generate
```

## Flag

Provides additional instructions.

Example:

```
-n
```

## Argument

Value provided to the flag.

Example:

```
MyProject
```

---

# Example Command

```
sf project generate -n MyProject
```

## Breakdown

Command:

```
sf project generate
```

Flag:

```
-n
```

Argument:

```
MyProject
```

---

# Scratch Org Creation Command

```
sf org create scratch -f project-scratch-def.json -d
```

# Explanation

## Command

```
sf org create scratch
```

## Flag

```
-f
```

References scratch org definition file.

## Argument

```
project-scratch-def.json
```

### Switch

```
-d
```

Sets scratch org as default.

---

## Running Commands

Press Enter after typing a command.

Successful execution:

- macOS/Linux shows $
- Windows shows >

---

## Navigation Commands

### Change Directory

```
cd Documents
```

### Create Directory

```
mkdir Resume
```

### Navigate Multiple Directories

#### macOS/Linux

```
cd ~/Documents/Resume
```

#### Windows

```
cd ~\Documents\Resume
```

## Go Back One Directory

```
cd ..
```

## Current Directory

### macOS/Linux

```
pwd
```

### Windows

```
cd
```

---

# Command History

## macOS/Linux

```
history
```

Rerun command:

```
!499
```

## Windows

Use:

```
F7
```

---

# Visual Studio Code (VS Code)

Recommended code editor for Salesforce development.

Benefits:

- Extensions
- Integrated terminal
- Apex support
- Debugging
- Syntax highlighting

---

## Salesforce Extensions for VS Code

Provides:

- Apex debugging
- Syntax highlighting
- Code completion
- Salesforce integration

Successor to:

```
Force.com IDE
```

---

## Node.js

JavaScript runtime used for development.

Install LTS version.

Check version:

```
node -v
```

---

## npm

Node Package Manager.
Used to:

- Install packages
- Update packages
- Manage dependencies

Check version:

```
npm -v
```

Update npm:

```
npm install npm@latest -g
```

---

# Salesforce CLI

Used for:

- Scratch org creation
- Metadata deployment
- Source tracking
- Salesforce DX development

Update command:

```
sf update
```

---

# Summary

In Day 6, I learned:

- Event-driven architecture
- Platform Events
- Event publishing and subscribing

- Apex triggers for platform events
- Testing platform events
- Flow and API integrations
- Pub/Sub API basics
- Command-Line Interface basics
- Shells and command structure
- Navigation commands
- VS Code setup
- Node.js and npm
- Salesforce CLI
- Salesforce development environment setup